

Graph Theory — Mini Project (Assignment 2)

Meaningful Relationships from Global Superstore Data & Complex Graph Queries

Name	Priya Jadhav
PRN	25039001021
Dataset	Global Superstore Dataset
Tool	Neo4j AuraDB
Maximum Marks	20
Date of Submission	6 September 2026

Dataset: Global Superstore — transactional retail data

1. Dataset Description

The Global Superstore dataset is a transactional retail dataset containing 51,290 order-line records from 2011 to 2014 across multiple global markets. Each row represents a product sold within an order and contains customer, product, geography, shipping and financial information.

Important attributes used in the graph model include Order.ID, Customer.ID, Product.ID, Category, Sub.Category, Segment, Sales, Profit, Quantity, Discount, Shipping.Cost, Ship.Mode, City, State, Country, Region and Market.

Column / Concept	Purpose
Customer.ID / Customer.Name	Identifies the customer and customer segment.
Order.ID / Order.Date	Identifies the transaction and its date.
Product.ID / Product.Name	Identifies the purchased product.
Category / Sub.Category	Groups products into meaningful product classes.
Segment	Consumer, Corporate and Home Office customer groups.
Sales / Profit	Financial measures used for graph-based business analysis.
City / Country / Region / Market	Geographical delivery information.

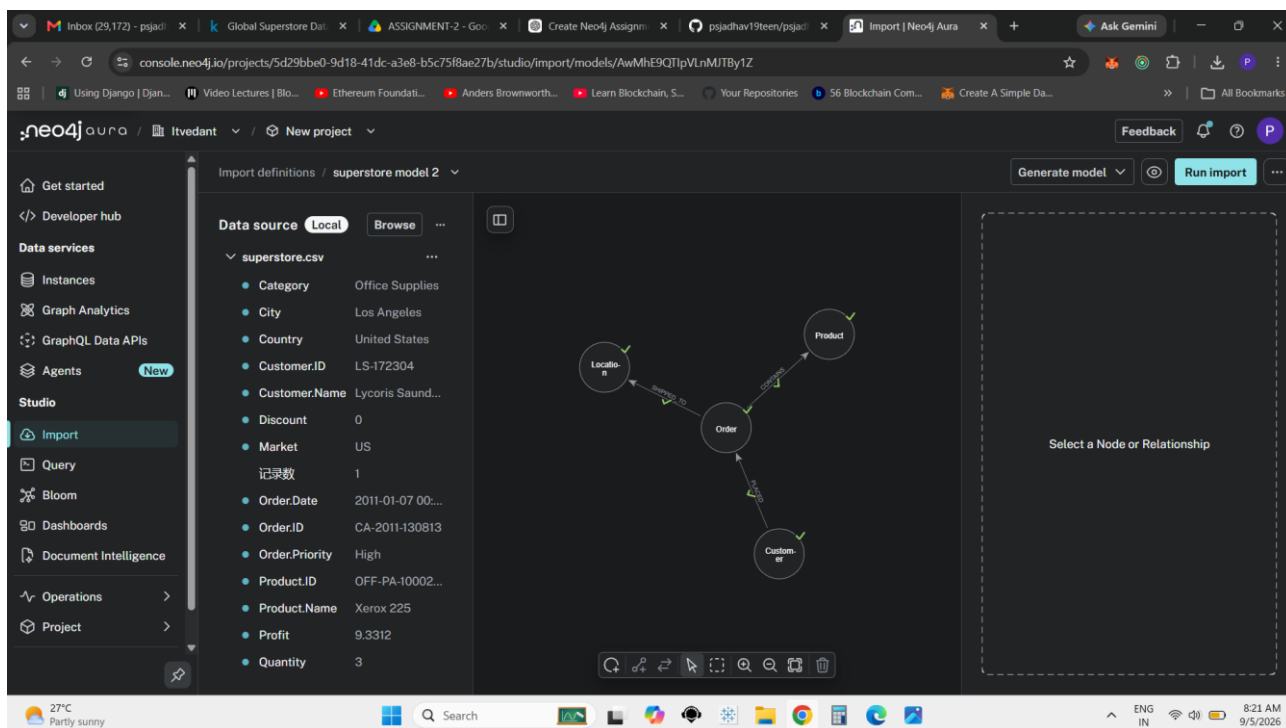


Figure 1 — Neo4j Import model showing Customer → Order → Product / Location and source properties.

2. Step 1 — Creating Meaningful Relationships and Mapping Parameters [5 Marks]

The dataset is modelled as a property graph representing the transaction lifecycle: who bought (Customer), through which transaction (Order), what was purchased (Product), and where it was delivered (Location).

2.1 Node Types

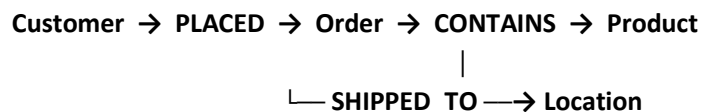
Node Label	Main Properties	Source Information
Customer	customerId, customerName, segment	Customer.ID, Customer.Name, Segment
Order	orderId, orderDate, shipDate, orderPriority	Order.ID, Order.Date, Ship.Date, Order.Priority
Product	productId, productName, category, subCategory	Product.ID, Product.Name, Category, Sub.Category
Location	city, state, country, region, market	City, State, Country, Region, Market

2.2 Relationship Types

From	Relationship	To	Meaning
Customer	PLACED	Order	Connects a customer to the order placed.
Order	CONTAINS	Product	Connects an order to the purchased product; transaction measures are available on the relationship.
Order	SHIPPED_TO	Location	Connects an order to its delivery location.

2.3 Graph Schema

The resulting meaningful path is:



```
5a962840$ MATCH (n) RETURN labels(n) AS NodeLabels, keys(n) AS Properties LIMIT 10;
```

Table	Raw
NodeLabels	Properties
1 ["Customer"]	["customerId", "customerName", "segment"]
2 ["Customer"]	["customerId", "customerName", "segment"]
3 ["Customer"]	["customerId", "customerName", "segment"]
4 ["Customer"]	["customerId", "customerName", "segment"]
5 ["Customer"]	["customerId", "customerName", "segment"]
6 ["Customer"]	["customerId", "customerName", "segment"]
7 ["Customer"]	["customerId", "customerName", "segment"]
8 ["Customer"]	["customerId", "customerName", "segment"]
9 ["Customer"]	["customerId", "customerName", "segment"]
10 ["Customer"]	["customerId", "customerName", "segment"]

Figure 2 — Neo4j property-key verification for the imported graph.

3. Step 2 — Import Data in Neo4j and Display Full Relationship [5 Marks]

The Global Superstore data was imported into Neo4j AuraDB using the Import facility. The current database contains four node labels and three relationship types.

Database Element	Count / Status
Customer nodes	4,873
Order nodes	25,035
Product nodes	10,292
Location nodes	3,636
Total nodes	43,836
Total relationships	102,729
Relationship types	CONTAINS, PLACED, SHIPPED_TO

3.1 Database Verification Query

```
MATCH (n)
RETURN labels(n)[0] AS Label, count(n) AS Count
ORDER BY Count DESC;
```

The screenshot displays the Neo4j AuraDB web interface. On the left, a sidebar contains navigation options like 'Get started', 'Developer hub', 'Data services', 'Instances', 'Graph Analytics', 'GraphQL Data APIs', 'Agents', 'Studio', 'Import', 'Query', 'Bloom', 'Dashboards', 'Document Intelligence', 'Operations', and 'Project'. The main panel is titled 'Database information' and shows 'Nodes (43,836)' with labels 'Customer', 'Location', 'Order', and 'Product'. It also shows 'Relationships (102,729)' with types 'CONTAINS', 'PLACED', and 'SHIPPED_TO'. Below this, 'Property keys' are listed, including 'category', 'city', 'country', 'customerId', 'customerName', 'discount', 'market', 'market2', 'orderDate', 'orderId', 'orderPriority', 'productId', 'productName', 'profit', 'quantity', 'region', 'rowId', 'sales', 'segment', and 'shipDate'. A query editor on the right contains the same Cypher query as shown in the text block. The query result is displayed in a table with two columns: 'Label' and 'Count'. The results are: 'Order' (25035), 'Product' (10292), 'Customer' (4873), and 'Location' (3636). The interface also shows a 'Run' button and a status bar at the bottom indicating the last update time as 8:28:55 AM.

Figure 3 — Neo4j database information and node-count verification.

3.2 Relationship-Type Verification

```
MATCH ()-[r]->()
RETURN DISTINCT type(r) AS RelationshipType
ORDER BY RelationshipType;
```

```
5a962840$ MATCH ()-[r]->() RETURN DISTINCT type(r) AS RelationshipType ORDER BY RelationshipType;
```

Table	Raw
RelationshipType	
1	"CONTAINS"
2	"PLACED"
3	"SHIPPED_TO"

Figure 4 — Verified relationship types: CONTAINS, PLACED and SHIPPED_TO.

3.3 Full Relationship Display

```
MATCH path =
  (c:Customer)-[:PLACED]->(o:Order)-[:CONTAINS]->(p:Product),
  (o)-[:SHIPPED_TO]->(l:Location)
RETURN path
LIMIT 25;
```

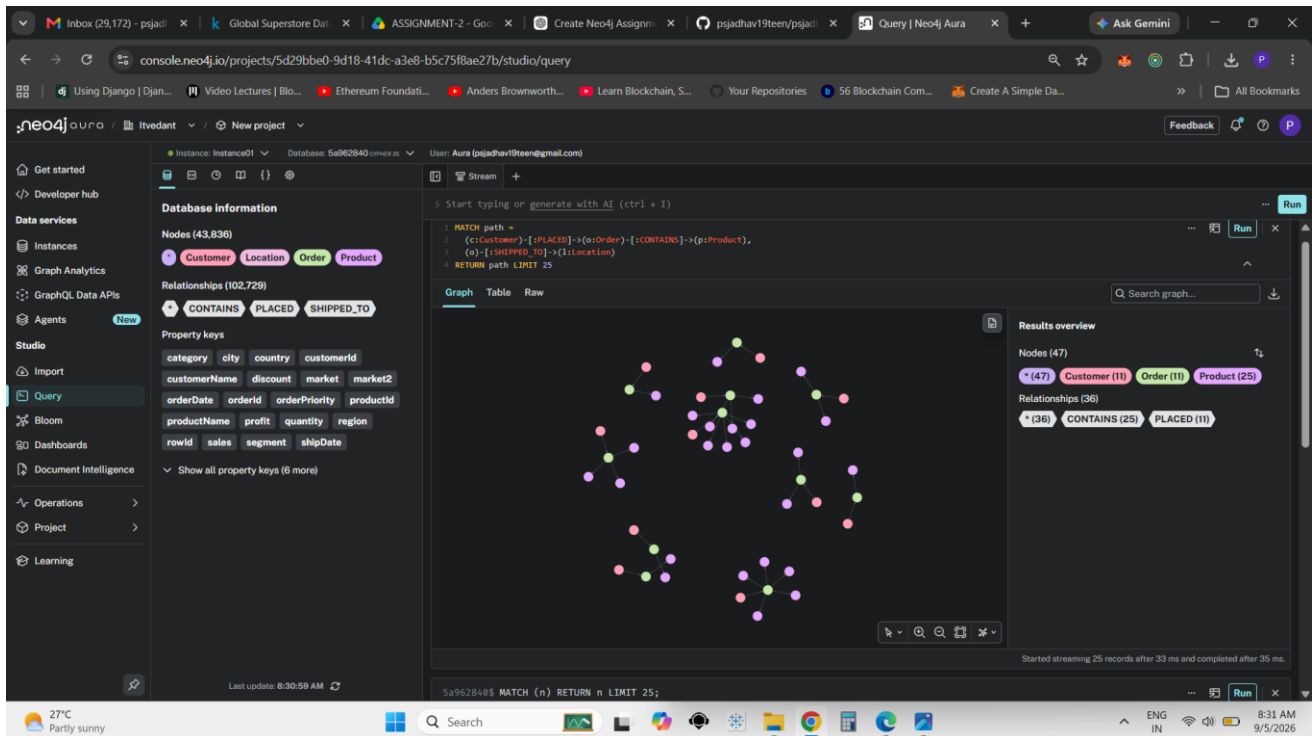


Figure 5 — Full graph visualization of Customer → Order → Product / Location relationships.

4. Step 3 — Two Complex Insights [10 Marks]

The two insights use graph traversal and aggregation over the Customer–Order–Product network. They are designed to extract information that is difficult to see from isolated tabular records.

4.1 Insight 1 — Product Centrality: Customer Reach + Sales + Profit [5 Marks]

Algorithm / Technique: Degree Centrality / Customer Reach Analysis. Products are ranked according to the number of distinct customers connected to them and the number of customer segments reached. Sales and profit are also aggregated to provide a richer business interpretation.

```
MATCH (c:Customer)-[:PLACED]->(o:Order)-[r:CONTAINS]->(p:Product)
WITH
  p.`productName` AS Product,
  p.category AS Category,
  p.subCategory AS SubCategory,
  count(DISTINCT c) AS UniqueCustomers,
  count(DISTINCT c.segment) AS SegmentsReached,
  round(sum(r.sales)) AS TotalSales,
  round(sum(r.profit)) AS TotalProfit
RETURN
  Product,
  Category,
  SubCategory,
  UniqueCustomers,
  SegmentsReached,
  TotalSales,
  TotalProfit
ORDER BY UniqueCustomers DESC, TotalSales DESC
LIMIT 15;
```

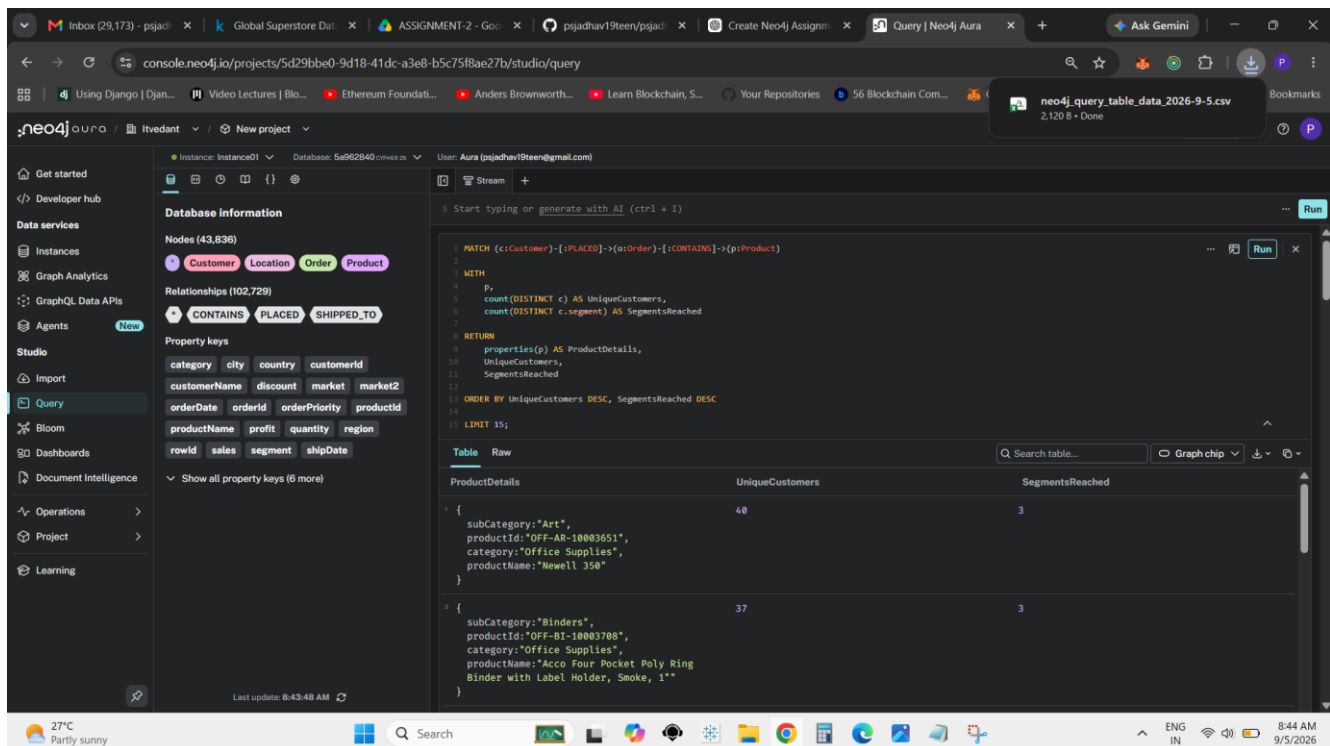


Figure 6 — Insight 1 result: products ranked by unique customer reach and segment reach.

Interpretation

The result identifies products with the widest customer reach in the graph. In the captured result, Newell 350 (Office Supplies, Art) reaches 40 unique customers and all 3 customer segments. Acco Four Pocket Poly Ring Binder with Label Holder, Smoke, 1" (Office Supplies, Binders) reaches 37 unique customers and all 3 segments. This indicates that these products have broad cross-segment penetration and are structurally important product nodes. The addition of Sales and Profit allows centrality to be considered together with financial contribution.

4.2 Insight 2 — Customer Segment × Product Category: Revenue + Profit Analysis [5 Marks]

Algorithm / Technique: Graph Aggregation / Community Profile Analysis. The query traverses Customer → Order → Product and groups the graph neighbourhood by customer segment and product category. It then compares customer count, product count, total sales and total profit.

```
MATCH (c:Customer)-[:PLACED]->(o:Order)-[r:CONTAINS]->(p:Product)
WITH
  c.segment AS Segment,
  p.category AS Category,
  count(DISTINCT c) AS Customers,
  count(DISTINCT p) AS Products,
  sum(r.sales) AS TotalSales,
  sum(r.profit) AS TotalProfit
RETURN
  Segment,
  Category,
  Customers,
  Products,
  round(TotalSales, 2) AS TotalSales,
  round(TotalProfit, 2) AS TotalProfit
ORDER BY TotalProfit DESC
LIMIT 10;
```

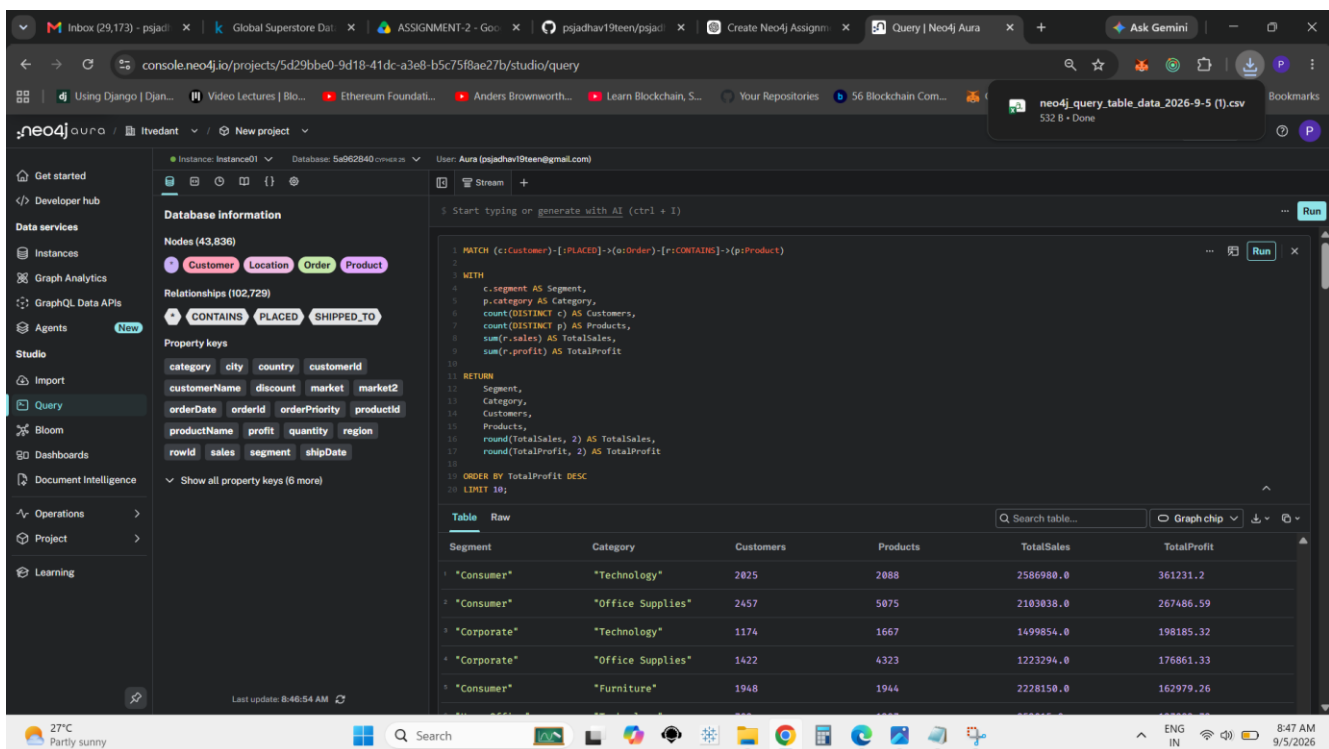


Figure 7 — Insight 2 result: Customer Segment × Product Category with Sales and Profit.

Interpretation

The captured result shows Consumer–Technology as the leading combination by total profit among the visible rows, with TotalSales of 2,586,980.0 and TotalProfit of 361,231.2. Consumer–Office Supplies follows with TotalSales of 2,103,038.0 and TotalProfit of 267,486.59. Corporate–Technology records 1,499,854.0 in TotalSales and 198,185.32 in TotalProfit. The graph therefore highlights Technology as an important category and shows that customer segment and category together provide a more informative view than revenue alone.

5. Additional Neo4j Evidence

The following screenshots document the current Neo4j environment and support the graph model, relationship structure and query execution used in the analysis.

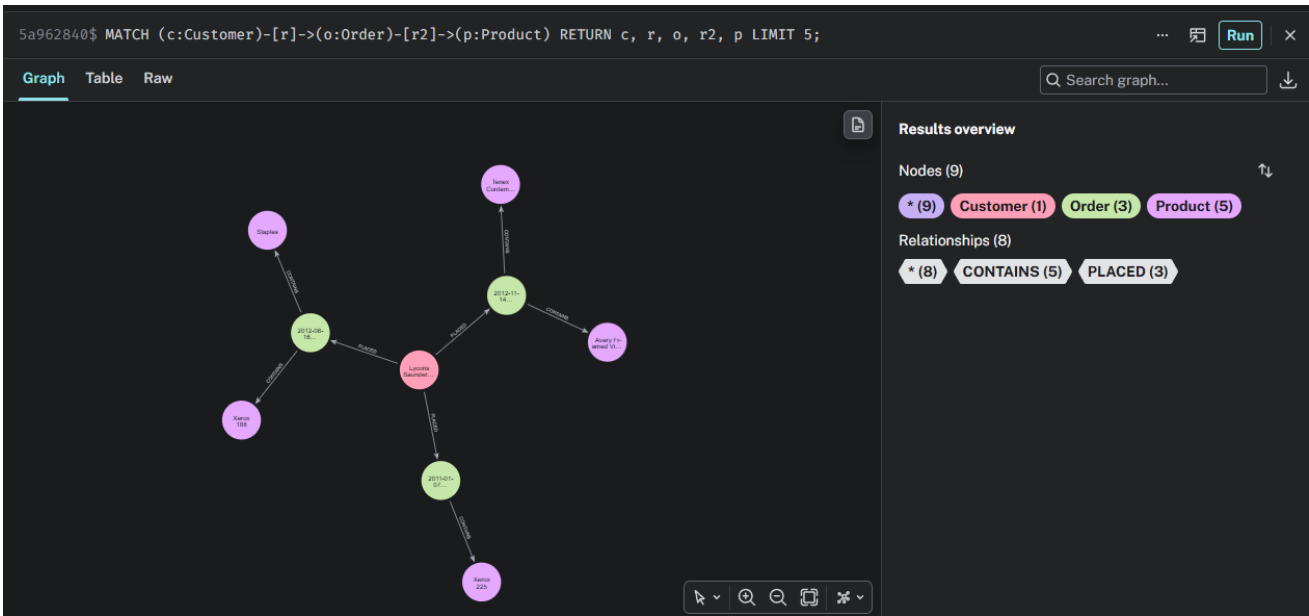


Figure 8 — Example Customer → Order → Product graph traversal.

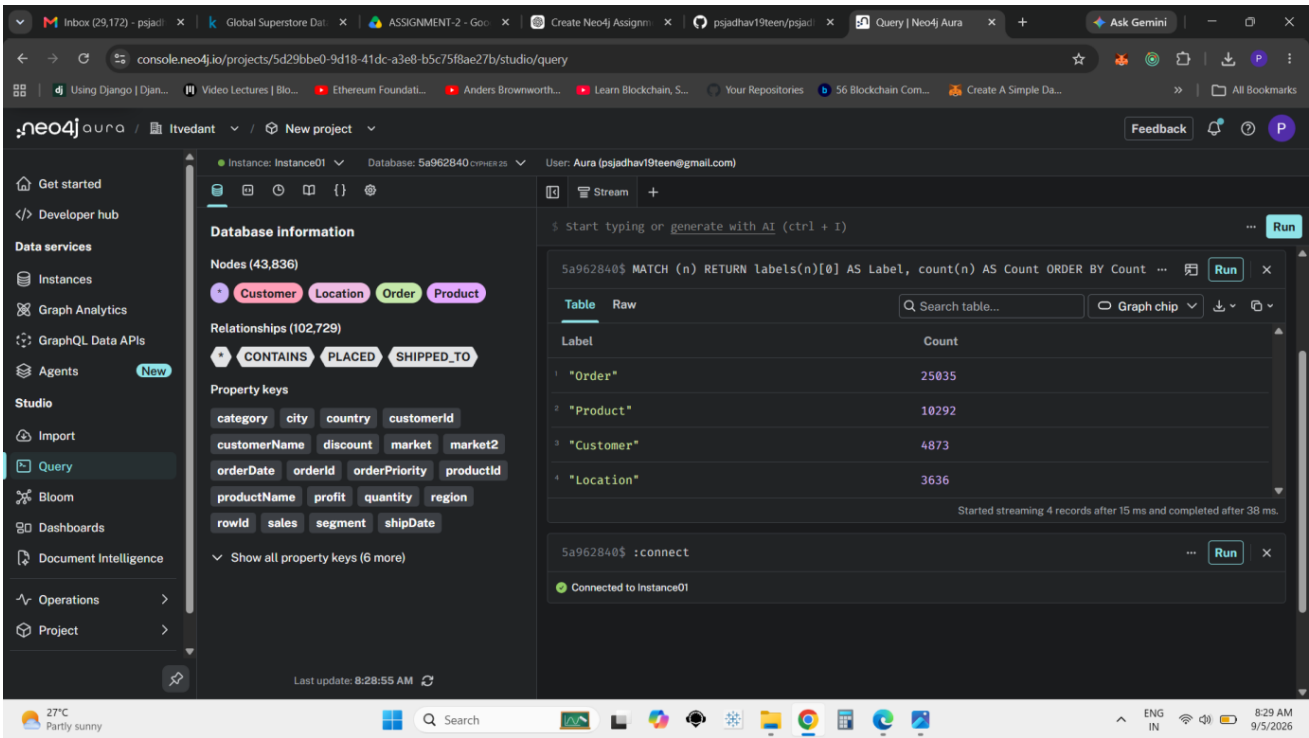


Figure 9 — Database information panel showing 43,836 nodes.

6. Conclusion

The Global Superstore dataset was successfully represented as a Neo4j property graph with Customer, Order, Product and Location nodes connected through PLACED, CONTAINS and SHIPPED_TO relationships. The graph structure makes it possible to analyse customer reach, product centrality, segment behaviour and financial performance through multi-hop traversal.

Insight 1 identified products with broad customer and segment reach, demonstrating how graph connectivity can reveal strategically important products. Insight 2 combined customer segment and product category relationships with sales and profit, revealing high-value combinations such as Consumer–Technology in the captured results. Together, the analyses demonstrate how graph-based queries can provide meaningful business insights from transactional data.

7. Key Queries Used

1. Node verification

```
MATCH (n)
RETURN labels(n)[0] AS Label, count(n) AS Count
ORDER BY Count DESC;
```

2. Relationship verification

```
MATCH ()-[r]->()
RETURN DISTINCT type(r) AS RelationshipType
ORDER BY RelationshipType;
```

3. Full relationship display

```
MATCH path =
  (c:Customer)-[:PLACED]->(o:Order)-[:CONTAINS]->(p:Product),
  (o)-[:SHIPPED_TO]->(l:Location)
RETURN path
LIMIT 25;
```